Ad Composer Lab: Requirements & Assignment Guide

Oct 17, 2026 · @Abinash

Students finish a FastAPI backend that turns a model photo, a product photo and a marketing quote into a finished advertisement image, using the same llm_client.py enterprise/direct switch as the sample scripts.

Problem and user story

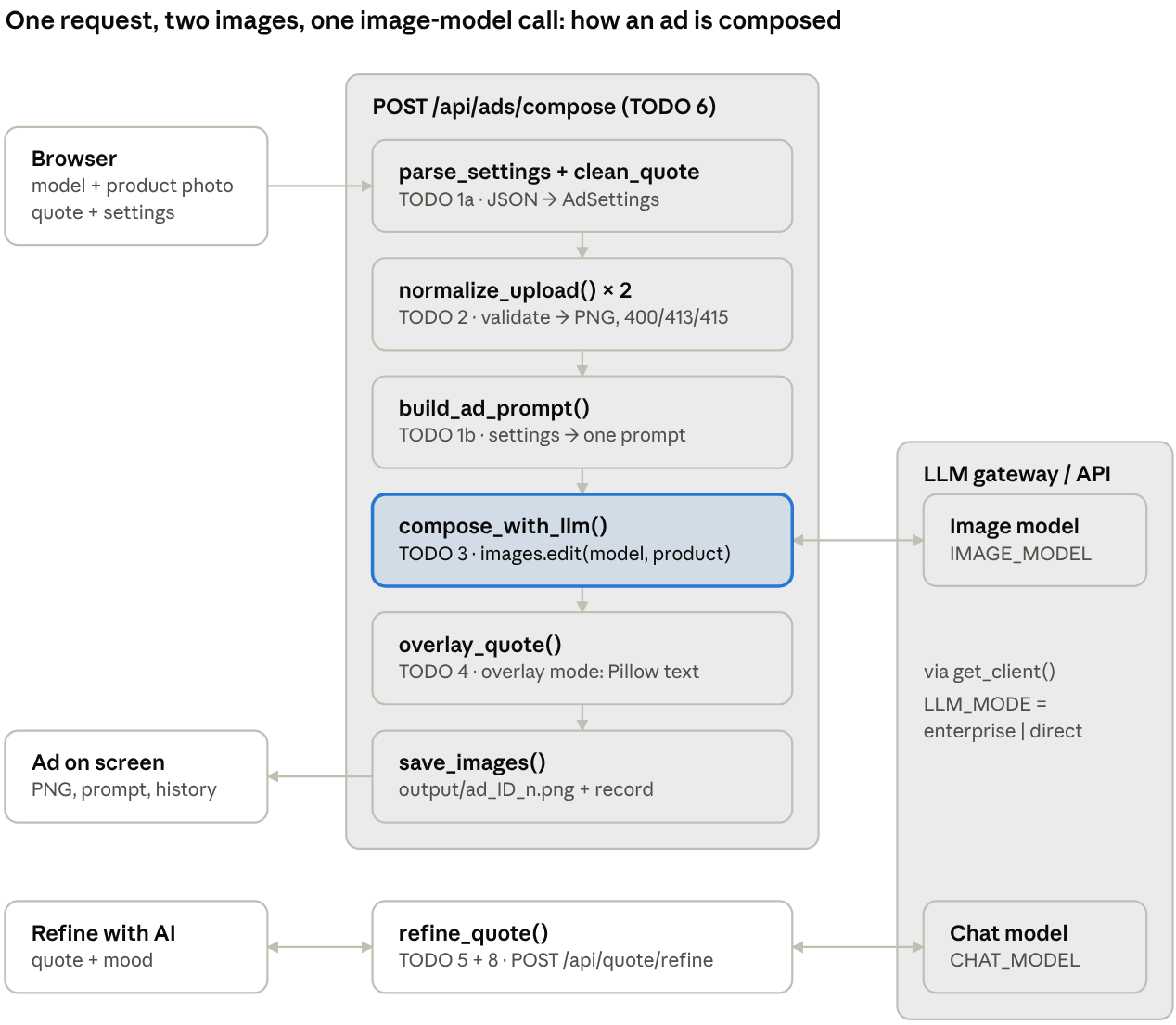
A marketing user should get a campaign-ready ad from three inputs, without a designer: **a model photo, a product photo and a one-line marketing quote.**

> *As a marketing executive, I upload a photo of our model and a photo of our product, type our headline, pick a look (style, scene, mood, format, text placement), and download a finished advertisement image in under two minutes.*

The app has two AI jobs and one deterministic job:

1. **Image composition** (image model, `IMAGE_MODEL`, default gpt-image-1): `images.edit` with both photos merges the person and the product into one styled scene.
2. **Text generation** (chat model, `CHAT_MODEL`): optional "Refine with AI" turns the user's line into three polished headlines.
3. **Text composition** (Pillow, no AI): in *overlay* mode the server writes the quote onto the image so spelling, font and colour are exact.

Architecture



request flow · browser → FastAPI → LLM gateway

Only `compose_with_llm()` and `refine_quote()` touch the gateway; every other step is plain Python, so the flow up to Checkpoint B runs with zero model calls.

Scope: provided vs to build

Students write about 150 lines of Python across 8 TODOs; everything else ships ready in the starter zip.

Part	Starter zip	Students build
`llm_client.py`, `00_check_setup.py`	Identical to the sample scripts	Nothing (do not edit)
`.env.example`	Sample settings + 4 app settings (`IMAGE_TIMEOUT_SECONDS`, `AD_DRY_RUN`, `MAX_UPLOAD_MB`, `IMAGE_INPUT_FIDELITY`)	Nothing
Frontend (`frontend/`)	Complete: uploads, quote, settings panel, prompt preview, results, history	Nothing
FastAPI shell (`app/main.py`, `config.py`, `schemas.py`)	Complete: app, error handlers, static hosting, Pydantic contract	Nothing
Read-only endpoints	`/api/health`, `/api/options`, `/api/ads`, `/api/ads/files/{name}`	Nothing
Prompt engineering (`prompt_builder.py`)	Preset sentences + safety rules	TODO 1a `clean_quote`, TODO 1b `build_ad_prompt`
Image service (`image_service.py`)	Dry-run collage, saving, error class	TODO 2 `normalize_upload`, TODO 3 `compose_with_llm`
Text composition (`text_overlay.py`)	Fonts + helpers	TODO 4a `wrap_text`, TODO 4b `overlay_quote`
Quote service (`quote_service.py`)	System prompt + JSON parser	TODO 5 `refine_quote`
Endpoints (`routes.py`)	Signatures + docstrings	TODO 6 compose, TODO 7 preview, TODO 8 refine
Tests (`tests/`)	47 tests with a fake LLM client	Make them pass

Until a TODO is done, its button in the UI answers **501 "TODO n: implement …"**, so students always know what blocks them.

Functional requirements

Each requirement is checked by the test suite or by a manual check in class.

ID	Requirement	How it is checked
FR-1	Accept exactly two images (model, product) as JPEG, PNG or WEBP, up to `MAX_UPLOAD_MB`	415 for other types, 413 when too large, 400 when empty, unreadable or under 64 px
FR-2	Normalise uploads: fix phone rotation (EXIF), keep transparency, downscale to 2048 px max, convert to PNG	`test_2_3_image_service.py`
FR-3	Accept a marketing quote up to 120 characters; required unless text mode is `none`	400 "Please enter a marketing quote"
FR-4	Turn every output setting into one structured prompt (task, subjects, scene, layout, text, brand, rules, extra)	`test_1_prompt_builder.py`; Preview prompt button
FR-5	Send both images to the image model in a fixed order: image 1 = model, image 2 = product	Test checks the `image=[...]` order
FR-6	Map aspect ratio, quality and variations onto the API call (`size` , `quality` , `n`)	Test checks the call kwargs
FR-7	Text modes: `ai` (model paints the quote), `overlay` (Pillow writes it), `none` (image only)	Overlay test checks pixels change only in the chosen band
FR-8	Return 1–3 PNG ads, save them to `output/` with a JSON record, serve them for download	`/api/ads/files/{name}` returns `image/png`

ID	Requirement	How it is checked
FR-9	Prompt preview endpoint returns the exact prompt with no model call	`test_prompt_preview`
FR-10	Refine-quote endpoint returns up to 3 de-duplicated headlines from the chat model	`test_quote_refine_*`
FR-11	Dry-run mode (`AD_DRY_RUN=true`) runs the full flow with a local collage and never calls the LLM	`test_compose_dry_run_never_calls_llm`
FR-12	Every error reaches the browser as JSON `{error, hint}` , never a stack trace	Manual: break `.env` , read the message

API contract

All endpoints live under `/api` ; Swagger at `/docs` lets students call each one by hand.

Method	Path	Request	Response	Status
POST	`/api/ads/compose`	multipart: `model_image` (file), `product_image` (file), `quote` (text), `settings` (AdSettings as a JSON string)	`{id, images: [{url, filename}], prompt, quote, settings, mode, model, elapsed_seconds}`	TODO 6
POST	`/api/prompt/preview`	JSON `{quote, settings}`	`{prompt, size, text_mode}`	TODO 7
POST	`/api/quote/refine`	JSON `{quote, mood, brand_name?}`	`{original, suggestions: [...]}`	TODO 8

Method	Path	Request	Response	Status
GET	`/api/health`	none	mode, gateway, model names, dry-run flag	Provided
GET	`/api/options`	none	allowed values + defaults for every setting	Provided
GET	`/api/ads`	`?limit=12`	recent ad records	Provided
GET	`/api/ads/files/{name}`	none	the PNG	Provided

Error format, every endpoint: `{"error": "what went wrong", "hint": "what to do"}`.

Status	When
400	Empty or unreadable image, image too small, missing quote, model rejected the request (incl. safety system)
413	Upload larger than `MAX_UPLOAD_MB`
415	Not JPEG / PNG / WEBP
422	A setting outside its allowed values
429	Gateway rate limit
501	TODO not implemented yet (starter only)
502	Gateway auth (401/403), connection failure, empty model answer
504	Image model slower than `IMAGE_TIMEOUT_SECONDS`

Output settings → prompt and API

The model never sees the dropdowns, only text: every setting must become a sentence in the prompt or a parameter on the call.

Setting	Allowed values	Default	Lands in
style	studio, luxury, minimal, vibrant, festive, lifestyle	studio	Prompt SCENE (preset sentence)
scene	auto, plain-studio, nature-outdoor, urban-street, beach, festive-lights, home-interior	auto	Prompt SCENE
mood	premium, playful, bold, calm, warm	premium	Prompt SCENE (lighting) and the refine-quote request
aspect_ratio	square, landscape, portrait	square	API `size` 1024x1024 / 1536x1024 / 1024x1536 + prompt LAYOUT
quality	low, medium, high	medium	API `quality` (use low in class)
text_mode	ai, overlay, none	ai	Prompt TEXT; overlay also runs Pillow
quote_position	top, center, bottom	bottom	Prompt LAYOUT/TEXT + overlay band
font_style	modern-sans, elegant-serif, bold-display	modern-sans	Prompt TEXT + overlay font file
text_color	#RRGGBB	#FFFFFF	Prompt TEXT + overlay text
accent_color	#RRGGBB	#E4572E	Prompt BRAND + overlay panel
brand_name	up to 40 chars, optional	none	Prompt BRAND + overlay sub-line
variations	1–3	1	API `n`
extra_instructions	up to 300 chars, optional	none	Prompt, **last**, labelled "must not override the rules above"

Prompt section order: **TASK → SUBJECTS → SCENE → LAYOUT → TEXT → BRAND → RULES → EXTRA**. The RULES section always carries six guard-rails: only the two input subjects, product label unchanged, model identity unchanged, family-friendly with no medical or financial claims, no watermarks or stray text, photorealistic finish.

Non-functional requirements

- **Enterprise and direct from one codebase.** Every model call goes through `get_client()` from the unchanged `llm_client.py`. `LLM_MODE=enterprise` uses MSAL client-credentials + FIL AI Gateway; `LLM_MODE=direct` uses an API key. No `OpenAI(...)` anywhere else; no model names in code (use `settings.image_model`, `settings.chat_model`).
- **Secrets.** `.env` is git-ignored; no keys or tokens in code, logs or responses. Inside FIL, only SRM-ticket credentials.
- **Cost control.** Validate uploads before the model call. Image calls use `max_retries=0` (a retry doubles the bill and the wait). Class default: `quality=low`, `variations=1`.
- **Timeouts.** Image calls use `IMAGE_TIMEOUT_SECONDS` (default 600, as in `07_image_edit_compose.py`); chat calls use `LLM_TIMEOUT_SECONDS`. Endpoints are sync `def` so FastAPI runs the blocking call in a worker thread.
- **Gateway compatibility.** Optional parameters (`input_fidelity`) are only sent when set in `.env`, because a gateway may reject unknown keys.
- **Offline development.** `AD_DRY_RUN=true` and the faked client in tests let the whole lab run without the gateway.
- **Safe file serving.** Downloads only serve `ad_*.png` from `output/`; no path traversal.
- **Responsible AI.** Use only images you have rights to; no colleague photos without consent; no misleading claims; label real campaigns as AI-generated. The prompt's rules are a floor, not a guarantee.

The 8 TODOs and checkpoints

Do them in this order; the first real model call happens only at Checkpoint C.

Order	TODO	File · function	Skill practised	Unlocks
1	1a	`prompt_builder.clean_quote`	Sanitise user text before it enters a prompt	—
2	1b	`prompt_builder.build_ad_prompt`	Settings → structured prompt, image order, text modes, guard-rails	—
3	7	`routes` POST `/api/prompt/preview`	First FastAPI endpoint with Pydantic	**A: Preview prompt works**
4	2	`image_service.normalize_upload`	Validating uploads with Pillow; 400/413/415	—
5	4a	`text_overlay.wrap_text`	Word-wrap by pixel width	—
6	4b	`text_overlay.overlay_quote`	Text + image composition (RGBA layer, alpha composite)	—
7	6	`routes` POST `/api/ads/compose`	Multipart endpoint that wires the pipeline	**B: dry-run ad with quote**
8	3	`image_service.compose_with_llm`	Multi-image `images.edit` via `get_client()`, timeout, error mapping	**C: first real ad**
9	5	`quote_service.refine_quote`	Chat call that must return JSON	—
10	8	`routes` POST `/api/quote/refine`	Thin endpoint over a service	**D: Refine with AI works**

Every TODO's docstring spells out the steps; `pytest tests/test_1_prompt_builder.py -v` (and the other files) shows progress one TODO at a time.

Definition of done and grading

A submission is done when all four boxes are ticked; it is graded out of 100.

- [ ] `pytest` shows **47 passed**
- [ ] Two real ads from the same inputs: one `text_mode=ai`, one `text_mode=overlay`
- [ ] No edits to `llm_client.py`, `.env.example`, `schemas.py`, `main.py`, frontend or tests
- [ ] Student can explain why `overlay` exists when the model can draw text

Area	Points	Full marks when
Tests	40	47/47 passing (pro rata otherwise)
Prompt quality	20	Clear sections; explicit image order; overlay mode leaves a band; rules present; extra instructions last and lower priority
Error handling	15	Uploads validated before the model call; SDK errors mapped to the right status + hint; no stack traces in the browser
Real output	15	Two real ads (ai vs overlay); difference explained
Code hygiene	10	Only `get_client()`; no hard-coded models or keys

Stretch goals: Hinglish quote in ai vs overlay mode; optional third image for a brand logo; cost guard refusing `quality=high` with 3 variations; A/B prompt versions; async job + polling instead of one long request.

Setup and run

Same setup as the sample scripts, then one `uvicorn` command; start in dry-run mode.

```
python -m venv .venv
# Windows: .venv\Scripts\activate     macOS/Linux: source .venv/bin/activate
pip install -r requirements.txt
copy .env.example .env        # macOS/Linux: cp .env.example .env
# edit .env: LLM_MODE=enterprise + FIL_* values from your SRM ticket
#            AD_DRY_RUN=true while building
python 00_check_setup.py      # must end with "All good"
uvicorn app.main:app --reload --port 8000
# app:     http://127.0.0.1:8000
# swagger: http://127.0.0.1:8000/docs
pytest                        # 46 failing at start, 47 passing when done
```

If `uvicorn` reports `ModuleNotFoundError: llm_client`, run it from the project folder (the one holding `.env`), not from `app/`.